

Advanced Linux Commands & Shell Scripting for DevOps

Operational handbook: advanced command-line tooling, package management, cron, systemd, diagnostics, and production-grade Bash scripting. Built from the *Linux Systems Engineering & DevOps Administration* handbook, extended with advanced commands, scripting patterns, and ready-to-use scripts.

Table of Contents

Part A — Advanced Linux Commands

1. [Package Installation & Management](#)
2. [Advanced File Search & Text Processing](#)
3. [Process, Resource & Signal Control](#)
4. [Advanced Networking Commands](#)
5. [Storage, Inodes & I/O](#)
6. [Cron Jobs — Complete Automation Guide](#)
7. [systemd: Services, Units, Timers & Journals](#)
8. [Diagnostic Command Reference](#)

Part B — Shell Scripting 9. [Shell Scripting Fundamentals](#) 10. [Control Flow: Conditions, Loops, case](#) 11. [Functions, Arrays & Parameter Expansion](#) 12. [Input, Arguments & Option Parsing](#) 13. [Production Scripting Patterns](#) 14. [Real-World DevOps Scripts](#) 15. [Debugging, Linting & Testing Scripts](#)

Part C — Reference 16. [Master Cheat Sheet](#) 17. [Common Pitfalls](#) 18. [Practice Exercises](#) 19. [Notes & Corrections on the Source Handbook](#)

Part A — Advanced Linux Commands

1. Package Installation & Management

Package managers automate software acquisition, cryptographic signature verification, dependency resolution, and upgrades. Provisioning tools (Ansible, cloud-init, Docker builds) depend directly on resilient package-manager commands.

1.1 Command comparison

Operation	Debian / Ubuntu (apt)	RHEL / Rocky / Fedora (dnf)	Alpine (apk)
Update catalog index	<code>sudo apt-get update -y</code>	<code>sudo dnf check- update</code>	<code>apk update</code>
Install	<code>sudo apt-get install -y <pkg></code>	<code>sudo dnf install -y <pkg></code>	<code>apk add --no- cache <pkg></code>
Remove/ purge	<code>sudo apt-get purge -y <pkg></code>	<code>sudo dnf remove -y <pkg></code>	<code>apk del <pkg></code>
Full upgrade	<code>sudo apt-get upgrade -y</code>	<code>sudo dnf upgrade -y</code>	<code>apk upgrade</code>
Clean cache	<code>sudo apt-get clean && rm -rf /var/lib/apt/lists/*</code>	<code>sudo dnf clean all</code>	<code>rm -rf /var/cache/apk/*</code>
List files of a package	<code>dpkg -L <pkg></code>	<code>rpm -ql <pkg></code>	<code>apk info -L <pkg></code>
Which package owns a file	<code>dpkg -S /usr/bin/curl</code>	<code>rpm -qf /usr/bin/curl</code>	<code>apk info -W /usr/bin/curl</code>
Search	<code>apt-cache search <kw></code>	<code>dnf search <kw></code>	<code>apk search <kw></code>
Package details	<code>apt-cache show <pkg></code>	<code>dnf info <pkg></code>	<code>apk info <pkg></code>
List installed	<code>dpkg -l</code> / <code>apt list -- installed</code>	<code>rpm -qa</code> / <code>dnf list installed</code>	<code>apk list -I</code>

Note: `apt-get` is preferred in **scripts** (stable output/interface); `apt` is meant for interactive use.

1.2 Standard installation sequence

Step 1 — Synchronize repository metadata

```
bash
```

```
sudo apt-get update -y
```

Step 2 — Install with non-interactive flags (prevents CI/CD pipeline hangs)

```
bash
```

```
sudo DEBIAN_FRONTEND=noninteractive apt-get install -y --no-install-recommends
```

- `-y` auto-answers yes.
- `DEBIAN_FRONTEND=noninteractive` suppresses interactive config dialogs.
- `--no-install-recommends` avoids pulling optional packages (smaller, safer).

Step 3 — Verify and clean

```
bash
```

```
which nginx && nginx -v
```

```
sudo apt-get clean && sudo rm -rf /var/lib/apt/lists/*
```

1.3 Dockerfile layer hygiene

Always consolidate `update`, `install`, and `clean` into a **single `RUN`** statement. Separate steps leave deleted archives permanently embedded in lower image layers.

```
dockerfile
```

```
# ✅ Good: one layer, cache cleaned in the same layer
```

```
RUN apt-get update \  
    && DEBIAN_FRONTEND=noninteractive apt-get install -y --no-install-recommends \  
        nginx curl jq \  
    && apt-get clean \  
    && rm -rf /var/lib/apt/lists/*
```

```
# ❌ Bad: cleanup in a later layer does NOT shrink the earlier layer
```

```
RUN apt-get update  
RUN apt-get install -y nginx  
RUN rm -rf /var/lib/apt/lists/*
```

Alpine equivalent: `RUN apk add --no-cache curl jq` (no separate cleanup needed).

1.4 Advanced package operations

```
bash
```

```

# Pin / hold a version (prevent surprise upgrades)
sudo apt-mark hold nginx
sudo apt-mark unhold nginx
apt-mark showhold
sudo apt-get install -y nginx=1.24.0-2ubuntu7          # exact version
apt-cache policy nginx                                # available versions + repository
apt-cache madison nginx

# RHEL family
sudo dnf install -y 'dnf-command(versionlock)'
sudo dnf versionlock add nginx
sudo dnf history                                       # transaction history
sudo dnf history undo <ID>                             # roll back a transaction
sudo dnf repoquery --installed --extras               # packages not in any repository

# Fix broken installs (Debian)
sudo dpkg --configure -a
sudo apt-get install -f
sudo apt-get autoremove -y --purge

# Download a package without installing
apt-get download curl
dnf download curl

# Inspect a .deb / .rpm before installing
dpkg -c package.deb ; dpkg -I package.deb
rpm -qpl package.rpm ; rpm -qpi package.rpm

```

Adding a third-party repository (modern, keyring-based, Debian/Ubuntu)

```

bash

curl -fsSL https://example.com/repo/key.gpg | sudo gpg --dearmor -o /etc/apt/keys/example.gpg
echo "deb [signed-by=/etc/apt/keys/example.gpg] https://example.com/repo stable main" | sudo tee /etc/apt/sources.list.d/example.list
sudo apt-get update

```

❗ Avoid the deprecated `apt-key add`. Scope each key to its repo with `signed-by=`.

Automatic security updates

```

bash

```

```
sudo apt-get install -y unattended-upgrades
sudo dpkg-reconfigure -plow unattended-upgrades
# RHEL: sudo dnf install -y dnf-automatic && sudo systemctl enable --now dnf-au
```

Common errors

Error	Fix
<code>Could not get lock</code> <code>/var/lib/dpkg/lock-frontend</code>	Another apt is running (e.g. unattended-upgrades). Wait, or <code>ps aux grep -E "apt dpkg"</code> . Don't delete lock files blindly.
<code>Unable to locate package</code>	Run <code>apt-get update</code> ; check repo/component (universe) and architecture
<code>NO_PUBKEY</code> / GPG error	Add the repo's signing key to a keyring
<code>held broken packages</code>	<code>apt-get install -f</code> , check holds and mixed repos

2. Advanced File Search & Text Processing

2.1 `find` — beyond the basics

```
bash

find /var/log -type f -name "*.log" -size +100M           # big logs
find . -type f -mtime +30 -delete                         # delete files
find . -type f -newer reference.txt                       # newer than a
find . -type f -mmin -10                                  # modified in
find . -type f \( -name "*.jpg" -o -name "*.png" \)       # OR condition
find . -type f ! -name "*.log"                           # NOT
find / -xdev -type f -perm -o+w 2>/dev/null               # world-writab
find / -xdev -type f \( -perm -4000 -o -perm -2000 \) 2>/dev/null # SUID/SGID a
find . -type f -user www-data
find . -empty -type d
find . -maxdepth 2 -type d
find . -type f -exec grep -l "TODO" {} +                 # '+' batches
find . -type f -print0 | xargs -0 -P4 -n50 gzip          # parallel, wh
```

Always use `-print0` with `xargs -0` when filenames may contain spaces or newlines.

2.2 `xargs` — build commands from input

```
bash
```

```
cat hosts.txt | xargs -I{} ssh {} uptime # one command per line
cat urls.txt | xargs -P8 -n1 curl -fsS -o /dev/null -w "%{http_code} %{url}\n"
find . -name "*.tmp" -print0 | xargs -0 rm -f
echo "a b c" | xargs -n1 # one arg per line
```

2.3 `grep` — advanced

```
bash

grep -E "ERROR|FATAL" app.log # extended regex
grep -P '\d{3}-\d{4}' file # Perl regex (lookaheads,
grep -oP 'user=\K\w+' app.log # print only the match af
grep -rIn --include="*.py" --exclude-dir=".git" "password" .
grep -w "root" /etc/passwd # whole word
grep -c "404" access.log # count
grep -m5 "error" app.log # stop after 5 matches
grep -B2 -A5 "Traceback" app.log # context
grep -vE '^\s*(#|$)' /etc/ssh/sshd_config # strip comments and blan
zgrep "error" /var/log/syslog.*.gz # search compressed logs
```

2.4 `awk` — the log-analysis workhorse

```
bash

awk '{print $1}' access.log # first field
awk -F: '{print $1, $3}' /etc/passwd # custom delimiter
awk '$9 >= 500' access.log # filter by field value
awk '{sum += $10} END {print sum/1024/1024 " MB"}' access.log
awk '{count[$1]++} END {for (ip in count) print count[ip], ip}' access.log | so
awk 'NR==1 || /ERROR/' app.log # header + error lines
awk -v threshold=80 '$5+0 > threshold {print $6, $5}' <(df -h | tr -d '%') #
awk '{printf "%-20s %s\n", $1, $9}' access.log
```

Typical pipeline — **top 10 client IPs with 5xx errors**:

```
bash

awk '$9 ~ /^5/ {c[$1]++} END {for (i in c) print c[i], i}' access.log | sort -r
```

2.5 `sed` — stream editing

```
bash
```

```

sed 's/foo/bar/g' file           # substitute (print)
sed -i.bak 's/foo/bar/g' file   # in-place with backup
sed -n '10,20p' file             # print lines 10-20
sed '/^#/d;/^$/d' file           # delete comments and blank lines
sed -n '/START/,/END/p' file     # print a range between patterns
sed 's/^\(.*\)$/"\1"/' file      # wrap each line in quotes
sed -E 's/([0-9]+)-([0-9]+)/\2-\1/' file # extended regex with capture groups
sed '3i\inserted line' file      # insert before line 3
sed '$a\appended at end' file    # append at end
sed 's|/old/path|/new/path|g' file # use | as delimiter when text contains |

```

2.6 Sort, set operations, and column tools

```

bash

sort -u file                     # unique sort
sort -k2,2nr file                # sort by column 2 numerically, reverse
sort -t, -k3 file.csv            # CSV, column 3
sort file | uniq -c | sort -rn   # frequency count
sort -V versions.txt             # version-aware (1.2 < 1.10)
comm -23 <(sort a.txt) <(sort b.txt) # lines only in a.txt
comm -12 <(sort a.txt) <(sort b.txt) # lines in both
diff -u old.conf new.conf        # unified diff
diff <(ssh web1 cat /etc/hosts) <(ssh web2 cat /etc/hosts)
paste -d, file1 file2            # merge side by side
column -t -s, data.csv           # pretty-print CSV
cut -d, -f1,3 data.csv           # extract columns
tr -s ' ' < file                 # squeeze repeated spaces
tr -d '\r' < windows.txt > unix.txt # strip CRs
rev ; tac file                   # reverse chars / reverse lines

```

`<( ... )` is **process substitution** — it presents command output as a temporary file.

2.7 JSON & YAML

```

bash

```

```

curl -s https://api.example.com/items | jq '.items[] | {id, name}'
jq -r '.items[].name' file.json # raw output, no quotes
jq '.[] | select(.status=="failed")' jobs.json
jq -r '.[] | [.id, .name] | @csv' data.json
jq '.spec.replicas = 5' deployment.json
kubectl get pods -o json | jq -r '.items[] | select(.status.phase!="Running") |
jq '.spec.template.spec.containers[0].image' deployment.yaml
jq -i '.spec.replicas = 3' deployment.yaml # in-place edit

```

2.8 `rsync` — advanced syncing

```

bash

rsync -avz --progress src/ user@host:/dest/
rsync -avz --delete --exclude='.git' --exclude='node_modules' src/ dest/
rsync -avzn --delete src/ dest/ # -n = dry run (ALWAYS pr
rsync -avz -e "ssh -i ~/.ssh/key -p 2222" src/ user@host:/dest/
rsync -a --link-dest=/backup/prev src/ /backup/$(date +%F)/ # space-efficient
rsync -avz --bwlimit=5000 src/ user@host:/dest/ # limit bandwidth (KB/s)

```

Trailing slash matters: `src/` copies contents; `src` copies the directory itself.

3. Process, Resource & Signal Control

3.1 Inspect

```

bash

ps -eo pid,ppid,user,%cpu,%mem,etime,stat,cmd --sort=-%mem | head -15
ps -ef --forest
pgrep -afl nginx
pidof nginx
pstree -ap <PID>
top -b -n1 | head -20 # batch mode (scripts)
htop # interactive
watch -n2 'ps aux --sort=-%cpu | head -5'
uptime ; nproc ; cat /proc/loadavg

```

Process states (`STAT` column)

State	Meaning
(R)	Running / runnable
(S)	Interruptible sleep
(D)	Uninterruptible sleep (usually waiting on I/O — can't be killed)
(Z)	Zombie (exited; parent hasn't reaped)
(T)	Stopped

3.2 Signals

Signal	Number	Use
(SIGHUP)	1	Reload config (many daemons), terminal hangup
(SIGINT)	2	Ctrl+C
(SIGQUIT)	3	Quit with core dump
(SIGKILL)	9	Force kill (cannot be caught)
(SIGTERM)	15	Graceful terminate (default for (kill))
(SIGSTOP) / (SIGCONT)	19 / 18	Pause / resume
(SIGUSR1) / (SIGUSR2)	10 / 12	App-defined (e.g. log reopen)

bash

```
kill -l                                # list signals
kill -TERM 1234 ; kill -HUP 1234 ; kill -9 1234
pkill -f "python3 worker.py"          # match full command line
killall -u deploy node
```

3.3 Priority, limits & containment

bash

```

nice -n 10 ./batch.sh                # start with lower priority (-20 high ...
renice 10 -p 1234
ionice -c3 -p 1234                    # idle I/O class
ionice -c2 -n7 tar -czf big.tgz /data # best-effort, lowest priority
timeout 30s ./slow_command            # kill after 30s (exit code 124)
timeout -k 5 30 ./cmd                 # SIGKILL 5s after SIGTERM if it persist

ulimit -a                             # shell limits
ulimit -n 65535                       # open files (current shell)
cat /proc/<PID>/limits                 # limits of a running process

# Run a command in a transient cgroup with resource caps (systemd)
sudo systemd-run --scope -p MemoryMax=512M -p CPUQuota=50% ./heavy_job.sh

```

3.4 Open files & sockets

```

bash

lsof -p <PID>                         # files opened by a process
lsof -i :8080                         # who uses the port
lsof -u deploy                        # files opened by a user
sudo lsof +L1                         # deleted-but-open files (hidden disk us
ls -l /proc/<PID>/fd | wc -l           # FD count
fuser -v 8080/tcp                    # process on a port

```

3.5 Background & persistent sessions

```

bash

nohup ./job.sh > job.log 2>&1 &
disown -h %1                          # detach job from shell
setsid ./job.sh &                     # new session
tmux new -s ops ; tmux ls ; tmux attach -t ops # detach with Ctrl+B then D

```

4. Advanced Networking Commands

```

bash

```

```

# Interfaces & routing
ip -br a                                # brief address view
ip -br link
ip route get 8.8.8.8                    # which route/interface is used
ip neigh                                # ARP table

# Sockets (ss replaces netstat)
sudo ss -tulpn                          # listening TCP/UDP + process
ss -tan state established '( dport = :443 )' # filter by state and port
ss -s                                  # summary
ss -tan | awk 'NR>1 {c[$1]++} END {for (s in c) print s, c[s]}' # connections

# Reachability
nc -zv 10.0.0.15 443                    # TCP port check
nc -zv -w3 host 20-25                  # scan a range with timeout (authorized
timeout 3 bash -c '</dev/tcp/10.0.0.15/443' && echo open || echo closed # no
mtr -rwc 20 example.com                # route + packet loss report

# DNS
dig +short example.com
dig +trace example.com                 # follow delegation chain
dig @1.1.1.1 example.com A
dig -x 8.8.8.8                        # reverse lookup
getent hosts example.com               # what the system resolver (nsswitch) re
resolvectl status

# HTTP timing breakdown
curl -o /dev/null -s -w "dns=%{time_namelookup} connect=%{time_connect} tls=%{t
curl -fsS --retry 5 --retry-delay 2 --retry-connrefused --max-time 10 https://s

# Packet capture
sudo tcpdump -i eth0 -nn port 443 and host 10.0.0.5
sudo tcpdump -i any -nn -c 100 -w /tmp/cap.pcap 'tcp port 80'
sudo tcpdump -i eth0 -A -s0 'tcp port 80 and (((ip[2:2] - ((ip[0]&0xf)<<2)) - (

# Firewall inspection
sudo iptables -L -n -v --line-numbers
sudo iptables -t nat -L -n -v
sudo nft list ruleset
sudo ufw status numbered

# TLS
echo | openssl s_client -connect example.com:443 -servername example.com 2>/dev

```

SSH power features

bash

```
ssh -L 5433:db.internal:5432 bastion      # local port-forward through bastion
ssh -J bastion user@10.0.1.20           # jump host
ssh -N -f -L 8080:localhost:80 user@host # background tunnel
ssh-add -l ; eval "$(ssh-agent -s)"
```

`~/.ssh/config` connection multiplexing (much faster repeated SSH/Ansible):

Host *

```
ControlMaster auto
ControlPath ~/.ssh/cm-%r@%h:%p
ControlPersist 10m
ServerAliveInterval 60
```

5. Storage, Inodes & I/O

bash

```
df -hT                                # usage + fs type
df -i                                 # INODE usage
du -xh --max-depth=1 / 2>/dev/null | sort -rh | head -15
ncdu -x /                             # interactive explorer
lsblk -f ; blkid ; findmnt
mount | grep -E "ro,|ro\)"            # read-only remounts (often after FS error)
```

Inode exhaustion — a disk can show free space yet fail with `No space left on device` if inodes are used up (millions of tiny files, e.g. session files, mail queues):

bash

```
df -i
# Find directories with the most files
for d in /var/*; do echo "$(find "$d" -xdev 2>/dev/null | wc -l) $d"; done | sort -nr
```

Hidden usage from deleted-but-open files

bash

```
sudo lsof +L1                         # restart/reload that process to release
```

I/O performance

```
bash

iostat -xz 1 5          # %util, await, r/s, w/s
iotop -oPa              # per-process I/O
vmstat 1 5              # wa = I/O wait
sudo hdparm -Tt /dev/sda # quick read speed test
dd if=/dev/zero of=/tmp/test bs=1M count=1024 oflag=direct status=progress #
fio --name=randrw --rw=randrw --bs=4k --size=1G --iodepth=16 --runtime=30 --tim
```

Mount options often used in production: `noatime` (reduce writes),
`nodelv,nosuid,noexec` (hardening for `/tmp`, `/var/tmp`), `nofail` (don't block boot).

LVM resize online

```
bash

sudo lvextend -L +10G -r /dev/vg0/data # -r grows the filesystem too
sudo xfs_growfs /data                  # if not using -r with XFS
```

6. Cron Jobs — Complete Automation Guide

Cron is the time-based job scheduler. It runs commands or scripts in the background at fixed intervals.

Primary DevOps applications

- Automated incremental and full backups
- Log rotation, compaction, and retention cleanup
- Periodic TLS certificate verification and renewal checks
- Database dumps and snapshot uploads to object stores
- Health checks, polling daemons, and alert monitors

6.1 Verify the daemon

```
bash
```

```
# Ubuntu / Debian
sudo systemctl enable --now cron
systemctl status cron

# RHEL / CentOS / Rocky
sudo systemctl enable --now crond
systemctl status crond
```

6.2 Crontab management

Command	Operation	Description
<code>crontab -e</code>	Edit	Edit current user's crontab
<code>crontab -l</code>	List	Show active jobs for the user
<code>crontab -r</code>	Remove	⚠ Deletes the whole crontab without prompting (use <code>crontab -ri</code> for a prompt)
<code>sudo crontab -u deploy -e</code>	Edit another user's	Manage a service account's crontab

Tip: back up before editing → `crontab -l > ~/crontab.bak`.

6.3 Time syntax

```
* * * * * command_to_execute
```

|

|

|

|

|

└─ Day of week (0-7; 0 and 7 = Sunday)

|

|

|

└─ Month (1-12 or JAN-DEC)

|

|

└─ Day of month (1-31)

|

└─ Hour (0-23)

└─ Minute (0-59)

Operators: `(*)` any · `()` list (`(1,15)`) · `-` range (`(1-5)`) · `/` step (`(*/10)`).

Common production patterns

Goal	Expression	Example
Every minute	<code>* * * * *</code>	<code>/opt/scripts/heartbeat.sh</code>
Every 5 minutes	<code>*/5 * * * *</code>	<code>/usr/bin/python3 /opt/metrics.py</code>
Every hour on the hour	<code>0 * * * *</code>	<code>/usr/bin/certbot renew --quiet</code>
Every night at 2:00 AM	<code>0 2 * * *</code>	<code>/home/vagrant/backup.sh</code>
Every Monday at 8:00 AM	<code>0 8 * * 1</code>	<code>/opt/reports/weekly_summary.sh</code>
Weekdays 9-17 every 15 min	<code>*/15 9-17 * * 1-5</code>	<code>/opt/scripts/poll.sh</code>
First day of month, 03:30	<code>30 3 1 * *</code>	<code>/opt/scripts/monthly.sh</code>
At system reboot	<code>@reboot</code>	<code>/opt/scripts/node_init.sh</code>

Shortcuts: `@hourly`, `@daily`, `@weekly`, `@monthly`, `@yearly`, `@reboot`.

Day-of-month and day-of-week are **OR-ed** when both are set (not AND). `0 0 13 * 5` runs on the 13th **and** on every Friday.

6.4 Step-by-step script automation

Step 1 — Author a production script (`~/backup.sh`)

```
bash

#!/bin/bash
set -eo pipefail

TIMESTAMP=$(date +%Y-%m-%d)
BACKUP_DIR="$HOME/backups"
SOURCE_DIR="$HOME/Documents"

echo "=====
echo "Backup started: $(date)"
mkdir -p "$BACKUP_DIR"
tar -czf "$BACKUP_DIR/home-{$TIMESTAMP}.tar.gz" -C "$HOME" Documents
echo "Backup completed successfully: $(date)"
echo "Archive: $BACKUP_DIR/home-{$TIMESTAMP}.tar.gz"
echo "=====
```

Step 2 — Make executable and test interactively

```
bash
```

```
chmod +x ~/backup.sh
```

```
~/backup.sh
```

```
ls -lh ~/backups/
```

Step 3 — Schedule with redirection and locking (`crontab -e`)

```
cron
```

```
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
```

```
SHELL=/bin/bash
```

```
MAILTO=""
```

```
0 2 * * * /usr/bin/flock -n /tmp/backup.lock /bin/bash /home/vagrant/backup.sh
```

6.5 DevOps cron rules

1. **Minimal environment:** cron does not load `~/.bashrc`. Declare `PATH` at the top of the crontab and use absolute paths.
2. **Overlap prevention:** use `flock -n` so a slow run doesn't stack with the next one.
3. **Stream redirection:** always `>> /path/to/log 2>&1` to retain logs.
4. **Escape `%`:** in crontab, `%` means newline — write `date +%F`.
5. **Make jobs idempotent** and safe to re-run.
6. **Monitor results:** alert on failure (e.g. ping a healthchecks URL on success: `&& curl -fsS https://hc-ping.com/<uuid>`)).

6.6 Other cron locations

Location	Notes
<code>/etc/crontab</code>	System crontab — has an extra user field
<code>/etc/cron.d/*</code>	Drop-in files (same format as <code>/etc/crontab</code> , with user field)
<code>/etc/cron.{hourly,daily,weekly,monthly}/</code>	Drop executable scripts here (no extensions in filenames on Debian)
<code>/etc/cron.allow</code> / <code>cron.deny</code>	Restrict who may use cron
<code>/var/spool/cron/</code>	Per-user crontabs (don't edit directly)

Debugging cron

```
bash
```

```
grep CRON /var/log/syslog          # Debian
journalctl -u cron -S today        # systemd
sudo tail -f /var/log/cron         # RHEL
env -i /bin/bash -c '/home/vagrant/backup.sh' # simulate cron's bare enviro
```

7. systemd: Services, Units, Timers & Journals

systemd is the init system and process supervisor on enterprise Linux. It manages process lifecycles, dependency graphs, and resource constraints through declarative unit files.

7.1 Essential commands

```
bash
```

```
# Lifecycle
```

```
sudo systemctl start <service>      # start immediately
sudo systemctl stop <service>       # SIGTERM, then SIGKILL after timeout
sudo systemctl restart <service>    # stop + start
sudo systemctl reload <service>     # re-read config, keep connections
sudo systemctl try-restart <service> # restart only if running
```

```
# Boot persistence
```

```
sudo systemctl enable <service>      # create symlinks for boot startup
sudo systemctl enable --now <service> # enable + start in one go
sudo systemctl disable <service>
sudo systemctl mask <service>        # make it impossible to start (links to
sudo systemctl unmask <service>
```

```
# Inspection
```

```
systemctl status <service>           # PID, state, memory, recent logs
systemctl is-active <service>        # active / inactive
systemctl is-enabled <service>       # enabled / disabled
systemctl --failed                   # failed units
systemctl list-units --type=service --state=running
systemctl list-unit-files --state=enabled
systemctl cat <service>              # show effective unit file (+ drop-ins)
systemctl show <service> -p MainPID,MemoryCurrent,ActiveState
systemctl list-dependencies <service>
sudo systemctl daemon-reload         # re-read unit files after edits
```

7.2 Creating a production-ready unit — step by step

Step 1 — Dedicated unprivileged service account (least privilege)

```
bash
```

```
sudo useradd --system --no-create-home --shell /usr/sbin/nologin appuser
```

Step 2 — Unit file `/etc/systemd/system/api-service.service`

```
ini
```

```
[Unit]
```

```
Description=Production API Service
```

```
Documentation=https://internal-wiki.company.local/docs/api
```

```
After=network-online.target remote-fs.target
```

```
Wants=network-online.target
```

```
[Service]
```

```
Type=simple
```

```
User=appuser
```

```
Group=appuser
```

```
WorkingDirectory=/var/www/api
```

```
ExecStart=/usr/bin/python3 -m uvicorn main:app --host 0.0.0.0 --port 8000
```

```
ExecReload=/bin/kill -s HUP $MAINPID
```

```
Restart=always
```

```
RestartSec=5s
```

```
# Security hardening
```

```
ProtectSystem=full
```

```
ProtectHome=true
```

```
NoNewPrivileges=true
```

```
PrivateTmp=true
```

```
# Resource boundaries
```

```
LimitNOFILE=65535
```

```
MemoryMax=1G
```

```
CPUQuota=150%
```

```
[Install]
```

```
WantedBy=multi-user.target
```

Step 3 — Register, enable, start

```
bash
```

```
sudo systemctl daemon-reload
sudo systemctl enable --now api-service.service
systemctl status api-service.service
```

Step 4 — Audit logs with journalctl

```
bash

journalctl -u api-service.service -f           # follow live
journalctl -u api-service.service -b           # since current boot
journalctl -u api-service.service -p err..emerg --no-pager  # errors only
```

7.3 Key unit directives

Directive	Meaning
<code>Type=simple</code>	Process started by <code>ExecStart</code> is the main process (default)
<code>Type=forking</code>	Service forks to background (needs <code>PIDFile=</code>)
<code>Type=oneshot</code>	Runs and exits (use with <code>RemainAfterExit=yes</code> or timers)
<code>Type=notify</code>	Service signals readiness via <code>sd_notify</code>
<code>Restart=always</code>	Restart on any exit; <code>on-failure</code> restarts only on non-zero/crash
<code>RestartSec=</code>	Delay between restarts
<code>StartLimitIntervalSec=</code> / <code>StartLimitBurst=</code>	Crash-loop protection (in <code>[Unit]</code> / <code>[Service]</code>)
<code>EnvironmentFile=</code>	Load <code>KEY=value</code> file (keep <code>chmod 600</code>)
<code>ExecStartPre=</code> / <code>ExecStartPost=</code>	Pre/post steps (e.g. config validation)
<code>TimeoutStopSec=</code>	Time to wait before SIGKILL
<code>After=</code> / <code>Requires=</code> / <code>Wants=</code>	Ordering vs. dependency (ordering $\neq$ dependency)
<code>MemoryMax=</code> / <code>CPUQuota=</code> / <code>TasksMax=</code>	cgroup limits
<code>ProtectSystem=strict</code> / <code>ReadWritePaths=</code>	Read-only filesystem except listed paths
<code>ProtectHome=</code> / <code>PrivateTmp=</code> / <code>NoNewPrivileges=</code>	Sandboxing
<code>ReadOnlyPaths=</code> / <code>InaccessiblePaths=</code>	Path-level restrictions

Tip: use `After=network-online.target` together with `Wants=network-online.target` when your service needs the network configured at start. `network.target` alone only means the networking *stack* has started, not that addresses are up.

7.4 Overrides & drop-ins (don't edit vendor units)

bash

```
sudo systemctl edit nginx          # creates /etc/systemd/system/nginx.service.d
sudo systemctl edit --full nginx    # full copy of the unit
sudo systemctl revert nginx         # remove overrides
```

Example override:

ini

```
[Service]
LimitNOFILE=100000
Restart=on-failure
```

7.5 Verify and score hardening

bash

```
systemd-analyze verify /etc/systemd/system/api-service.service
systemd-analyze security api-service.service      # exposure score + suggestions
systemd-analyze blame | head                     # slow boot units
systemd-analyze critical-chain
```

7.6 systemd timers (cron alternative)

/etc/systemd/system/backup.service

ini

```
[Unit]
Description=Nightly backup

[Service]
Type=oneshot
User=backup
ExecStart=/opt/scripts/backup.sh
```

/etc/systemd/system/backup.timer

ini

```
[Unit]
Description=Run backup nightly

[Timer]
OnCalendar=*-*-* 02:00:00
RandomizedDelaySec=300
Persistent=true

[Install]
WantedBy=timers.target
```

```
bash

sudo systemctl daemon-reload
sudo systemctl enable --now backup.timer
systemctl list-timers --all
systemd-analyze calendar "Mon..Fri 09:00"      # validate calendar expressions
journalctl -u backup.service -n 50
```

Advantages over cron: logs in the journal, dependencies, resource limits, `Persistent=true` to catch up after downtime, randomized delay to avoid thundering herd.

7.7 journalctl cookbook

```
bash

journalctl -u nginx -f                        # follow
journalctl -u nginx --since "1 hour ago"
journalctl -u nginx --since "2026-10-01 08:00" --until "2026-10-01 09:00"
journalctl -p err -b                          # errors this boot
journalctl -b -1                             # previous boot
journalctl -k                                # kernel messages
journalctl _PID=1234
journalctl -o json-pretty -n 1
journalctl -u app -g "timeout|refused"        # grep (regex)
journalctl --disk-usage
sudo journalctl --vacuum-time=14d
sudo journalctl --vacuum-size=500M
```

Make the journal persistent across reboots: `sudo mkdir -p /var/log/journal && sudo systemctl restart systemd-journald`.

8. Diagnostic Command Reference

Beyond service administration, production work requires port auditing, process troubleshooting, and capacity monitoring.

Domain	Command	Purpose
Network sockets	<code>sudo ss -tulpn</code>	Audit listening TCP/UDP ports and owning PIDs (replaces <code>netstat</code>)
Network probing	<code>nc -zv 10.0.0.15 443</code>	Check firewall reachability and TCP handshake without client overhead
Filesystem inodes	<code>df -i</code>	Inode consumption — a disk can show free capacity yet be full if millions of tiny files exhaust inodes
Memory & OOM	<code>sudo dmesg -T grep -i oom</code>	Check whether the kernel's OOM killer terminated your daemon
Storage I/O	<code>iostat -xz 1 5</code>	Device saturation (<code>%util</code>) and I/O wait times to find slow disk queues
Load	<code>uptime</code> , <code>nproc</code>	Compare load average to CPU core count
Memory	<code>free -h</code> , <code>vmstat 1 5</code>	<code>available</code> memory; swap activity (<code>si</code>)/(<code>so</code>)
Per-process stats	<code>pidstat -urd 1</code>	CPU, memory, and I/O per process
Open files	<code>lsof -p PID</code>	What a process has open
Syscalls	<code>strace -f -p PID</code>	What a hung process is doing
Kernel log	<code>dmesg -T tail -50</code>	Hardware/FS/OOM errors
Service health	<code>systemctl --failed</code>	All failed units
Failed logins	<code>lastb head</code> / <code>grep "Failed password" /var/log/auth.log</code>	Brute-force detection

Rapid triage script (paste on a sick server)

```
bash
```

```
uptime; echo; free -h; echo; df -hT -x tmpfs -x devtmpfs; echo; df -i | head -5
ps aux --sort=-%cpu | head -6; echo
ps aux --sort=-%mem | head -6; echo
ss -s; echo
systemctl --failed --no-pager; echo
dmesg -T | tail -15
```

Part B — Shell Scripting

9. Shell Scripting Fundamentals

9.1 Your first script

```
bash
```

```
#!/usr/bin/env bash
# hello.sh – prints a greeting
name="${1:-World}"
echo "Hello, ${name}!"
```

```
bash
```

```
chmod +x hello.sh
./hello.sh Safdar          # run via shebang
bash hello.sh              # run explicitly (no exec bit needed)
```

- The **shebang** (`#!/usr/bin/env bash`) selects the interpreter. (`#!/bin/sh`) means POSIX sh (no arrays, no `[[ ]]`).
- Scripts run in a **child process**: variables set inside don't persist in your shell. Use `source script.sh` (or `. script.sh`) to run in the current shell.

9.2 Variables & quoting

```
bash
```

```

name="Safdar"           # NO spaces around =
readonly PI=3.14         # constant
export APP_ENV=production # visible to child processes
unset name

echo "$name"             # double quotes: expand variables
echo '$name'             # single quotes: literal
echo "${name}_backup"    # braces to delimit variable name
echo "Today is $(date +%F)" # command substitution
echo "2+3 = $((2+3))"    # arithmetic

```

Rule #1: quote your variables — `"$var"`. Unquoted variables undergo word splitting and globbing, a top cause of destructive bugs (`rm -rf $DIR/*` with empty `DIR`).

9.3 Special variables

Variable	Meaning
<code>\$0</code>	Script name
<code>\$1 ... \$9</code> , <code>\${10}</code>	Positional arguments
<code>\$#</code>	Number of arguments
<code>"\$@"</code>	All args, each as a separate word (use this)
<code>"\$*"</code>	All args as one string
<code>\$?</code>	Exit status of last command
<code>\$\$</code>	PID of current shell/script
<code>\$_</code>	PID of last background job
<code>\$_</code>	Last argument of previous command
<code>\$RANDOM</code>	Random integer 0–32767
<code>\$SECONDS</code>	Seconds since script started
<code>\$LINENO</code> , <code>\$FUNCNAME</code> , <code>\$BASH_SOURCE</code>	Debug/introspection info

9.4 Exit codes

`0` = success, non-zero = failure. Conventions: `1` general error, `2` misuse, `126` not executable, `127` not found, `130` Ctrl+C, `124` timeout.

bash

```
grep -q "error" app.log
```

```
echo $?
```

```
exit 3
```

```
# 0 if found, 1 if not
```

```
# leave script with code 3
```

9.5 Redirection & pipes

bash

```
cmd > out.txt
```

```
# stdout overwrite
```

```
cmd >> out.txt
```

```
# append
```

```
cmd 2> err.txt
```

```
# stderr
```

```
cmd > all.log 2>&1
```

```
# both to same file (order matters!)
```

```
cmd &> all.log
```

```
# bash shorthand
```

```
cmd > /dev/null 2>&1
```

```
# discard all
```

```
cmd < input.txt
```

```
cmd1 | cmd2
```

```
# pipe
```

```
cmd | tee out.log
```

```
# screen + file
```

```
cmd |& tee all.log
```

```
# include stderr in pipe (bash)
```

Here-documents & here-strings

bash

```
cat > /etc/myapp.conf <<EOF
```

```
env=${APP_ENV}
```

```
port=8080
```

```
EOF
```

```
cat <<'EOF'
```

```
# quoted delimiter: NO expansion
```

```
literal $HOME stays literal
```

```
EOF
```

```
grep -c "x" <<< "$variable"
```

```
# here-string
```

9.6 Command chaining

bash

```

make && make install           # run second only if first succeeds
cmd || echo "failed"          # run only if first fails
cd /opt/app || exit 1         # classic guard
mkdir -p dir && cd dir
cmd1; cmd2                     # unconditional sequence
( cd /tmp && ls )              # subshell: cd doesn't affect parent
{ echo a; echo b; } > out.txt  # group commands, share redirection

```

10. Control Flow: Conditions, Loops, `case`

10.1 Conditionals

```

bash

if [[ -f /etc/nginx/nginx.conf ]]; then
    echo "nginx config exists"
elif [[ -d /etc/apache2 ]]; then
    echo "apache found"
else
    echo "no web server"
fi

```

Use `[[ ... ]]` in Bash (safer: no word splitting, supports `&&`, `||`, `=~`, pattern matching).

Use `[ ... ]` only for POSIX `sh`.

File tests

Test	True when
<code>-e f</code>	exists
<code>-f f</code>	regular file
<code>-d f</code>	directory
<code>-L f</code>	symlink
<code>-r</code> / <code>-w</code> / <code>-x</code>	readable / writable / executable
<code>-s f</code>	exists and non-empty
<code>f1 -nt f2</code> / <code>-ot</code>	newer / older than

String tests: `-z "$s"` (empty), `-n "$s"` (non-empty), `"$a" == "$b"`, `"$a" != "$b"`, `"$s" == prefix*` (glob), `"$s" =~ ^[0-9]+$` (regex). **Numeric tests:** `-eq` `-ne` `-gt` `-ge` `-lt` `-le`

`lt -le`), or `(( a > b ))`.

bash

```
if [[ $COUNT -gt 10 && "$ENV" == "prod" ]]; then echo "big prod"; fi
if (( COUNT % 2 == 0 )); then echo "even"; fi
if [[ "$EMAIL" =~ ^[^\@]+\@[^\@]+\.[a-z]+$ ]]; then echo "valid-ish"; fi
if command -v docker >/dev/null 2>&1; then echo "docker installed"; fi
if systemctl is-active --quiet nginx; then echo "nginx up"; fi
```

10.2 Loops

bash

```

# for over a list
for host in web1 web2 web3; do
    echo "== $host =="; ssh "$host" uptime
done

# for over files (glob – don't parse `ls`)
for f in /var/log/*.log; do
    [[ -e "$f" ]] || continue          # handle "no match" case
    gzip -k "$f"
done

# C-style
for ((i=1; i<=5; i++)); do echo "$i"; done

# brace expansion
for n in {1..5}; do echo "$n"; done
mkdir -p /srv/{app,logs,backup}

# while with condition
count=0
while (( count < 3 )); do ((count++)); echo "$count"; done

# read file line by line (safe form)
while IFS= read -r line; do
    echo "line: $line"
done < file.txt

# read command output without subshell variable loss
while IFS= read -r pod; do echo "$pod"; done < <(kubectl get pods -o name)

# until
until curl -fsS http://localhost:8000/health >/dev/null; do
    echo "waiting for service..."; sleep 2
done

# loop control
for i in {1..10}; do
    (( i == 3 )) && continue
    (( i == 6 )) && break
    echo "$i"
done

```

10.3 `case`

bash

```
case "$1" in
  start)    systemctl start myapp ;;
  stop)     systemctl stop myapp ;;
  restart|reload) systemctl restart myapp ;;
  status)   systemctl status myapp ;;
  *)        echo "Usage: $0 {start|stop|restart|status}" >&2; exit 2 ;;
esac
```

10.4 Arithmetic

bash

```
a=7; b=3
echo $((a + b)) $((a * b)) $((a / b)) $((a % b)) $((a ** 2))
((a++)) ; ((a += 5))
# floating point: use bc or awk
echo "scale=2; 10/3" | bc
awk 'BEGIN {printf "%.2f\n", 10/3}'
```

11. Functions, Arrays & Parameter Expansion

11.1 Functions

bash

```
log() {
    # arguments: $1, $2 ... ; "$@" for all
    local level="$1"; shift          # 'local' keeps variables function-scoped
    printf '%s [%s] %s\n' "$(date '+%F %T')" "$level" "$*"
}

die() { log ERROR "$*" >&2; exit 1; }

is_root() { [[ $EUID -eq 0 ]]; }

get_ip() { hostname -I | awk '{print $1}'; } # "return" values via stdout
ip=$(get_ip)

is_root || die "Run as root"
```

- `return N` sets the function's exit status (0-255). Output goes via `echo` and is captured with `$(...)`.

- Put reusable functions in a library: `source ./lib/common.sh`.

11.2 Indexed arrays

```
bash

servers=(web1 web2 web3)
servers+=(web4)           # append
echo "${servers[0]}"      # first
echo "${servers[@]}"      # all elements
echo "${#servers[@]}"     # count
echo "${servers[@]:1:2}"  # slice
for s in "${servers[@]"; do echo "$s"; done    # ALWAYS quote "${arr[@]}"
unset 'servers[1]'

mapfile -t lines < file.txt      # read file into array (one element per l
```

11.3 Associative arrays (Bash 4+)

```
bash

declare -A ports=( [http]=80 [https]=443 [ssh]=22)
ports[postgres]=5432
echo "${ports[https]}"
for name in "${!ports[@]"; do echo "$name -> ${ports[$name]}"; done
[[ -v ports[ssh] ]] && echo "ssh defined"
```

11.4 Parameter expansion (very handy)

Syntax	Result
<code>\${var:-default}</code>	Use <code>default</code> if <code>var</code> unset/empty
<code>\${var:=default}</code>	Same, and assign it
<code>\${var:?message}</code>	Abort with message if unset/empty
<code>\${var:+alt}</code>	<code>alt</code> if <code>var</code> is set, else empty
<code>\${#var}</code>	Length
<code>\${var:2:5}</code>	Substring (offset 2, length 5)
<code>\${var#prefix}</code> / <code>\${var##prefix}</code>	Remove shortest / longest prefix match
<code>\${var%suffix}</code> / <code>\${var%%suffix}</code>	Remove shortest / longest suffix match
<code>\${var/old/new}</code> / <code>\${var//old/new}</code>	Replace first / all
<code>\${var^^}</code> / <code>\${var,,}</code>	Upper / lower case

```
bash
```

```
file="/var/log/nginx/access.log.gz"
echo "${file##*/}"      # access.log.gz  (basename)
echo "${file%/*}"       # /var/log/nginx (dirname)
echo "${file%.gz}"      # /var/log/nginx/access.log
echo "${file%%.*}"      # /var/log/nginx/access
PORT="${PORT:-8080}"
: "${DB_HOST:?DB_HOST must be set}" # fail fast on missing config
```

12. Input, Arguments & Option Parsing

12.1 Reading input

```
bash
```

```
read -r -p "Continue? [y/N] " answer
[[ "$answer" =~ ^[Yy]$ ]] || exit 0

read -r -s -p "Password: " pw ; echo # -s hides typing
read -r -t 10 -p "Reply within 10s: " reply # timeout
```

12.2 Positional arguments

bash

```
[[ $# -ge 2 ]] || { echo "Usage: $0 <env> <version>" >&2; exit 2; }
env="$1"; version="$2"
shift 2
echo "extra args: $@"
```

12.3 `getopts` — short options

bash

```
usage() {
    cat <<EOF
Usage: $0 [-e env] [-v version] [-n] [-h]
    -e environment (default: staging)
    -v version to deploy (required)
    -n dry-run
    -h help
EOF
}

ENV="staging"; DRY_RUN=false; VERSION=""
while getopts ":e:v:nh" opt; do
    case "$opt" in
        e) ENV="$OPTARG" ;;
        v) VERSION="$OPTARG" ;;
        n) DRY_RUN=true ;;
        h) usage; exit 0 ;;
        :) echo "Option -$OPTARG needs an argument" >&2; exit 2 ;;
        \?) echo "Unknown option -$OPTARG" >&2; usage; exit 2 ;;
    esac
done
shift $((OPTIND - 1))
[[ -n "$VERSION" ]] || { usage; exit 2; }
```

12.4 Long options

bash

```

while [[ $# -gt 0 ]]; do
  case "$1" in
    --env)      ENV="$2"; shift 2 ;;
    --env=*)    ENV="${1#*=}"; shift ;;
    --dry-run)  DRY_RUN=true; shift ;;
    -h|--help)  usage; exit 0 ;;
    --)         shift; break ;;
    -*)         echo "Unknown option: $1" >&2; exit 2 ;;
    *)         break ;;
  esac
done

```

13. Production Scripting Patterns

13.1 Strict mode

```

bash

#!/usr/bin/env bash
set -Eeuo pipefail
IFS=$'\n\t'

```

Flag	Effect
<code>-e</code>	Exit immediately when a command fails
<code>-u</code>	Treat unset variables as errors
<code>-o pipefail</code>	A pipeline fails if any stage fails
<code>-E</code>	ERR trap is inherited by functions/subshells

Caveats: `set -e` doesn't trigger inside `if`/`&&`/`||` conditions; use `cmd || true` where a failure is acceptable. `grep` returns 1 on "no match", which can abort scripts under `-e` — handle explicitly: `grep -q x f || true`.

13.2 Logging

```

bash

```

```
LOG_FILE="${LOG_FILE:-/var/log/myscript.log}"
log() { printf '%s [%s] %s\n' "$(date '+%F %T')" "$1" "${*:2}" | tee -a "$LOG_
info() { log INFO "$@"; }
warn() { log WARN "$@"; }
err() { log ERROR "$@"; }
die() { err "$@"; exit 1; }

# Send to syslog/journald
logger -t myscript -p user.info "Backup finished"
```

Send logs to **stderr** and data results to **stdout**, so scripts compose in pipelines.

13.3 Cleanup & error handling with `trap`

```
bash

TMP_DIR="$(mktemp -d)"

cleanup() {
    local rc=$?
    rm -rf "$TMP_DIR"
    (( rc != 0 )) && echo "Script failed with exit code $rc" >&2
    exit "$rc"
}

trap cleanup EXIT
trap 'echo "Error on line $LINENO: $BASH_COMMAND" >&2' ERR
trap 'echo "Interrupted" >&2; exit 130' INT TERM
```

13.4 Locking (prevent concurrent runs)

```
bash

LOCK_FILE="/var/lock/myjob.lock"
exec 9>"$LOCK_FILE"
flock -n 9 || { echo "Another instance is running" >&2; exit 1; }
# ... critical section; lock released when script exits
```

Or from cron: `flock -n /tmp/job.lock /opt/job.sh`.

13.5 Retry with exponential backoff

```
bash
```

```

retry() {
    local max="${1}" delay="${2}"; shift 2
    local attempt=1
    until "$@"; do
        if (( attempt >= max )); then
            echo "Failed after $attempt attempts: $*" >&2
            return 1
        fi
        echo "Attempt $attempt failed; retrying in ${delay}s..." >&2
        sleep "$delay"
        delay=$(( delay * 2 ))
        (( attempt++ ))
    done
}

retry 5 2 curl -fsS http://localhost:8000/health

```

13.6 Idempotency & dry-run

```

bash

run() {
    # wrap mutating commands
    if [[ "$DRY_RUN" == true ]]; then echo "[dry-run] $*"; else "$@"; fi
}

run mkdir -p /opt/myapp # safe to repeat
id deploy &>/dev/null || run useradd -m deploy # create only if missing
grep -qxF 'net.ipv4.ip_forward=1' /etc/sysctl.conf || echo 'net.ipv4.ip_forward

```

13.7 Safety checks

```

bash

```

```
# Require root
[[ $EUID -eq 0 ]] || { echo "Run as root" >&2; exit 1; }

# Require commands
for cmd in curl jq tar; do
    command -v "$cmd" >/dev/null || { echo "Missing dependency: $cmd" >&2; exit 1; }
done

# Never rm -rf an empty/unset variable
rm -rf -- "${BUILD_DIR:?}/*"

# Validate input
[[ "$PORT" =~ ^[0-9]+$ ]] && (( PORT >= 1 && PORT <= 65535 )) || die "Invalid port"

# Safe temp files
tmp=$(mktemp)          # not /tmp/myfile.$$
```

13.8 Configuration & secrets

```
bash

# Load a .env file safely (KEY=value lines)
set -a; source /etc/myapp/app.env; set +a
chmod 600 /etc/myapp/app.env
```

- Never hardcode secrets; take them from the environment, a secrets manager (Vault, AWS Secrets Manager), or CI secret store.
- Avoid passing secrets as command-line arguments (visible in `ps`).
- Don't `set -x` around commands handling secrets.

13.9 Script template (copy/paste)

```
bash
```

```
#!/usr/bin/env bash
#
# name.sh - short description
# Usage: name.sh [-n] [-h] <arg>
#
set -Eeuo pipefail
IFS=$'\n\t'

readonly SCRIPT_NAME="$(basename "$0")"
readonly LOG_FILE="${LOG_FILE:-/tmp/${SCRIPT_NAME%.sh}.log}"
DRY_RUN=false

log() { printf '%s [%s] %s\n' "$(date '+%F %T')" "$1" "${*:2}" | tee -a "$LOG_
die() { log ERROR "$@"; exit 1; }
usage(){ echo "Usage: $SCRIPT_NAME [-n] [-h] <arg>"; }
run() { if $DRY_RUN; then log DRY "$*"; else "$@"; fi; }

cleanup() { ;; } # remove temp files, release resources
trap cleanup EXIT

main() {
    while getopts ":nh" o; do
        case "$o" in n) DRY_RUN=true ;; h) usage; exit 0 ;; *) usage; exit 2 ;; esac
    done
    shift $((OPTIND-1))
    [[ $# -ge 1 ]] || { usage; exit 2; }

    log INFO "Starting with arg: $1"
    # ... work here ...
    log INFO "Done"
}

main "$@"
```

14. Real-World DevOps Scripts

14.1 Backup with rotation and integrity check

```
bash
```

```
#!/usr/bin/env bash
set -Eeuo pipefail

SRC="${1:-/opt/myapp}"
DEST="/backup"
KEEP_DAYS=7
STAMP="$(date +%F_%H-%M-%S)"
ARCHIVE="$DEST/$(basename "$SRC")_${STAMP}.tar.gz"

mkdir -p "$DEST"
echo "[$(date)] Backing up $SRC → $ARCHIVE"

tar -czf "$ARCHIVE" -C "$(dirname "$SRC")" "$(basename "$SRC")"
tar -tzf "$ARCHIVE" >/dev/null # verify archive readable
sha256sum "$ARCHIVE" > "$ARCHIVE.sha256"

find "$DEST" -type f -name "$(basename "$SRC")_*.tar.gz*" -mtime +"$KEEP_DAYS"
echo "[$(date)] Done. Size: $(du -h "$ARCHIVE" | cut -f1)"
```

14.2 Service health check with auto-restart and alert

```
bash
```

```
#!/usr/bin/env bash
set -uo pipefail

SERVICE="${1:?Usage: $0 <service> <health-url>}"
URL="${2:?Usage: $0 <service> <health-url>}"
WEBHOOK="${SLACK_WEBHOOK:-}"

notify() {
    [[ -n "$WEBHOOK" ]] || return 0
    curl -fsS -X POST -H 'Content-Type: application/json' \
        -d '{"text": "${hostname}: $1\\"}' "$WEBHOOK" >/dev/null || true
}

if ! systemctl is-active --quiet "$SERVICE"; then
    logger -t healthcheck "$SERVICE not active - restarting"
    systemctl restart "$SERVICE" && notify "$SERVICE was down; restarted" || noti
    exit 0
fi

code=$(curl -s -o /dev/null -w '%{http_code}' --max-time 5 "$URL" || echo 000)
if [[ "$code" != "200" ]]; then
    logger -t healthcheck "$SERVICE unhealthy (HTTP $code) - restarting"
    systemctl restart "$SERVICE"
    notify "$SERVICE unhealthy (HTTP $code); restarted"
fi
```

Cron: `*/2 * * * * /usr/bin/flock -n /tmp/hc.lock /opt/scripts/healthcheck.sh myapp http://localhost:8000/health >> /var/log/healthcheck.log 2>&1`

14.3 Disk and inode usage alert

```
bash
```

```
#!/usr/bin/env bash
set -euo pipefail
THRESHOLD="${THRESHOLD:-85}"

df -P -x tmpfs -x devtmpfs -x overlay | awk 'NR>1 {print $5, $6}' | while read
usage="${pct%\\%}"
if (( usage >= THRESHOLD )); then
    echo "WARNING: $mount is at ${usage}% (threshold ${THRESHOLD}%)"
fi
done

df -Pi -x tmpfs -x devtmpfs -x overlay | awk 'NR>1 && $5+0 >= 85 {print "WARNIN
```

14.4 Log cleanup / retention

```
bash

#!/usr/bin/env bash
set -euo pipefail
LOG_DIR="${1:-/var/log/myapp}"
COMPRESS_AFTER=1      # days
DELETE_AFTER=30       # days

find "$LOG_DIR" -type f -name "*.log" -mtime +"$COMPRESS_AFTER" -exec gzip -f {} \;
find "$LOG_DIR" -type f -name "*.log.gz" -mtime +"$DELETE_AFTER" -print -delete
```

14.5 TLS certificate expiry checker

```
bash

#!/usr/bin/env bash
set -euo pipefail
WARN_DAYS=21

while read -r host; do
    [[ -z "$host" || "$host" == \#* ]] && continue
    end=$(echo | openssl s_client -connect "${host}:443" -servername "$host" 2>/dev/null
        | openssl x509 -noout -enddate 2>/dev/null | cut -d= -f2) || end=""
    if [[ -z "$end" ]]; then echo "ERROR $host: could not read certificate"; continue
    days=$(( ( $(date -d "$end" +%s) - $(date +%s) ) / 86400 ))
    if (( days < 0 )); then echo "EXPIRED $host ($days days)"
    elif (( days < WARN_DAYS )); then echo "WARN $host expires in $days days"
    else echo "OK $host ($days days)"; fi
done < "${1:-domains.txt}"
```

14.6 Zero-downtime-style deploy with symlink releases and rollback

```
bash
```

```
#!/usr/bin/env bash
set -Eeuo pipefail

APP="myapp"
BASE="/opt/$APP"
RELEASES="$BASE/releases"
CURRENT="$BASE/current"
VERSION="${1:?Usage: $0 <version>}"
NEW="$RELEASES/$VERSION"
KEEP=5

PREV="$(readlink -f "$CURRENT" || true)"

rollback() {
    echo "Deploy failed – rolling back" >&2
    if [[ -n "$PREV" && -d "$PREV" ]]; then
        ln -sfn "$PREV" "$CURRENT"
        sudo systemctl restart "$APP"
    fi
}
trap rollback ERR

mkdir -p "$NEW"
tar -xzf "/tmp/${APP}-${VERSION}.tar.gz" -C "$NEW"
ln -sfn "$NEW" "$CURRENT" # atomic switch
sudo systemctl restart "$APP"

# Health check with retries
for i in {1..10}; do
    if curl -fsS --max-time 3 http://localhost:8000/health >/dev/null; then
        echo "Healthy after $i checks"
        break
    fi
    [[ $i -eq 10 ]] && { echo "Health check failed"; false; }
    sleep 2
done

trap - ERR

# Prune old releases, keeping the newest $KEEP
ls -ldt "$RELEASES"/* | tail -n +$((KEEP + 1)) | xargs -r rm -rf
echo "Deployed $VERSION"
```

14.7 Bulk user provisioning from CSV

`users.csv`: `username,full name,group`

bash

```
#!/usr/bin/env bash
set -euo pipefail
[[ $EUID -eq 0 ]] || { echo "Run as root" >&2; exit 1; }

while IFS=, read -r user fullname group; do
    [[ -z "$user" || "$user" == "username" ]] && continue
    getent group "$group" >/dev/null || groupadd "$group"
    if id "$user" &>/dev/null; then
        echo "SKIP $user exists"
    else
        useradd -m -s /bin/bash -c "$fullname" -G "$group" "$user"
        passwd -l "$user" >/dev/null # lock password; use SSH keys
        install -d -m 700 -o "$user" -g "$user" "/home/$user/.ssh"
        echo "ADDED $user"
    fi
done < "${1:-users.csv}"
```

14.8 Run commands on many servers in parallel

bash

```
#!/usr/bin/env bash
set -uo pipefail
HOSTS_FILE="${1:?hosts file}"; shift
CMD="$*"

run_on() {
    local host="$1"
    out=$(ssh -o BatchMode=yes -o ConnectTimeout=5 "$host" "$CMD" 2>&1) \
    && printf '[OK] %s\n%s\n' "$host" "$out" \
    || printf '[FAIL] %s\n%s\n' "$host" "$out"
}

export -f run_on; export CMD

xargs -a "$HOSTS_FILE" -P 10 -I{} bash -c 'run_on {}'
```

Usage: `./pssh.sh hosts.txt "uptime; df -h /"`. (For serious fleets, prefer Ansible.)

14.9 Docker cleanup

bash

```
#!/usr/bin/env bash
set -euo pipefail
docker container prune -f --filter "until=72h"
docker image prune -a -f --filter "until=168h"
docker builder prune -f --filter "until=168h"
docker system df
```

14.10 One-page system report

```
bash

#!/usr/bin/env bash
echo "== $(hostname) @ $(date) =="
echo "OS:      $(. /etc/os-release && echo "$PRETTY_NAME")"
echo "Kernel:  $(uname -r)"
echo "Uptime:   $(uptime -p)"
echo "Load:     $(cut -d' ' -f1-3 /proc/loadavg) (cores: $(nproc))"
echo "Memory:   $(free -h | awk '/Mem:/ {print $3 " used / " $2 " total, " $7 " '
echo "Disk /:    $(df -h / | awk 'NR==2 {print $3 " used / " $2 " (" $5 ")}')")"
echo "IPs:       $(hostname -I)"
echo "Failed units:"; systemctl --failed --no-legend --no-pager | sed 's/^/ /'
echo "Top CPU:"; ps -eo pid,comm,%cpu --sort=-%cpu | sed -n '2,4p' | sed 's/^/ '
echo "Top MEM:"; ps -eo pid,comm,%mem --sort=-%mem | sed -n '2,4p' | sed 's/^/ '
```

15. Debugging, Linting & Testing Scripts

15.1 Debugging

```
bash

bash -n script.sh          # syntax check only (no execution)
bash -x script.sh          # trace every command
set -x ; ... ; set +x      # trace only a section
export PS4='+ ${BASH_SOURCE}:${LINENO}: ${FUNCNAME[0]}:-main}: ' # richer tracing
trap 'echo "ERR at line $LINENO: $BASH_COMMAND (exit $?)" >&2' ERR
echo "DEBUG: var=[$var]" >&2 # bracket values to expose stray whitespace
```

15.2 Linting & formatting

```
bash
```

```
sudo apt-get install -y shellcheck shfmt
shellcheck script.sh          # catches quoting bugs, unsafe patterns, portab
shfmt -i 2 -w script.sh       # auto-format (2-space indent)
```

Run ShellCheck in CI and as a pre-commit hook. Disable a rule only with justification: `(# shellcheck disable=SC2086)`.

15.3 Testing

- **bats-core** — Bash Automated Testing System:

```
bash

# test/backup.bats
@test "backup creates an archive" {
  run ./backup.sh /tmp/testdata
  [ "$status" -eq 0 ]
  ls /backup/testdata_*.tar.gz
}
```

- Test in **containers** (`(docker run --rm -v "$PWD":/w ubuntu:24.04 bash /w/script.sh)`) to simulate clean machines.
- Test the **failure paths** (missing dependency, no permission, full disk) — not just the happy path.
- Add `(--dry-run)` support and use it in staging.

15.4 Performance & portability tips

- Avoid unnecessary subprocesses in loops (`($(...))`, `(cat file | ...)`). Use Bash built-ins and parameter expansion.
 - For big text jobs prefer `(awk)`/`(sed)`/`(grep)` over a Bash `(while read)` loop.
 - If portability to Alpine/BusyBox matters, target POSIX `(sh)`: no arrays, no `([[ ]])`, no `(<( ))`; or install bash.
 - GNU vs BSD differences (`(sed -i)`, `(date -d)`, `(stat)`) matter on macOS.
-

Part C — Reference

16. Master Cheat Sheet

Packages

```
apt-get update && apt-get install -y --no-install-recommends PKG && apt-get clean
dnf install -y PKG          apk add --no-cache PKG
dpkg -L PKG | dpkg -S FILE | apt-mark hold PKG | dnf versionlock add PKG
```

Text & search

```
find . -type f -mtime +30 -delete          find . -print0 | xargs -0 -P4 CMD
grep -rIn --include="*.py" PATTERN .       grep -oP 'key=\K\w+' file
awk '{c[$1]++} END{for(k in c) print c[k],k}' f | sort -rn
sed -i.bak 's/old/new/g' file              sort | uniq -c | sort -rn
jq -r '.items[].name'                     comm -23 <(sort a) <(sort b)
```

Processes

```
ps aux --sort=-%mem | head                pgrep -af NAME          kill -TERM PID          kill -9
PID
timeout 30 CMD                            nice -n 10 CMD          ionice -c3 CMD          lsof -i :PORT
lsof +L1
```

Network

```
ss -tulpn      nc -zv HOST PORT      dig +short NAME      curl -fsS --max-time 5
URL
tcpdump -i any -nn port 443      mtr -rwc 20 HOST      ip -br a      ip route get
IP
```

Disk

```
df -hT      df -i      du -xh --max-depth=1 / | sort -rh | head      iostat -xz 1 5
lsblk -f
```

Cron

```
crontab -e|-l|-r      m h dom mon dow cmd      */5 * * * *      @reboot
flock -n /tmp/x.lock CMD >> /var/log/x.log 2>&1      # escape % as \%
```

systemd

```
systemctl start|stop|restart|reload|enable --now|disable|mask SVC
systemctl status|is-active|is-enabled|--failed|cat SVC      systemctl daemon-
reload
journalctl -u SVC -f | -b | -p err | --since "1 hour ago"
systemd-analyze verify|security|blame
```

Bash essentials

```
#!/usr/bin/env bash      set -Eeuo pipefail      trap cleanup EXIT      "${var:-
default}"
[[ -f f ]]  [[ $a -gt $b ]]  [[ $s =~ regex ]]      for x in "${arr[@]}"; do ...;
done
while IFS= read -r l; do ...; done < file      local x=...      getopts
":a:b" o
$? $# "$@" $0 $$ $!      mktemp -d      flock -n 9      shellcheck script.sh
```

17. Common Pitfalls

Pitfall	Why it hurts	Fix
Unquoted <code>\$var</code>	Word splitting and globbing; <code>rm -rf \$DIR/*</code> with empty <code>DIR</code> wipes <code>/*</code>	Quote: <code>"\$var"</code> ; use <code>\${DIR:?}</code>
<code>for f in \$(ls)</code>	Breaks on spaces/newlines	<code>for f in *; do</code> or <code>find -print0</code>
<code>cat file while read</code> and then using variables after	Pipe runs the loop in a subshell, variables vanish	<code>while ... done < file</code> or <code>< <(cmd)</code>
<code>cd dir; rm -rf *</code>	If <code>cd</code> fails you delete the wrong directory	<code>cd dir exit 1</code>
Assuming <code>set -e</code> catches everything	Ignored in <code>if</code> , <code>&&</code> / <code> </code> chains, and some subshell cases	Explicit checks; <code>trap ERR</code>
<code>[\$a = \$b]</code> with empty values	Syntax error	<code>[["\$a" == "\$b"]]</code>
<code>echo</code> with flags/escapes	Inconsistent across shells	<code>printf '%s\n' "\$x"</code>
Spaces around <code>=</code> in assignments	<code>name = x</code> runs a command named <code>name</code>	<code>name=x</code>
Cron job can't find commands	Minimal <code>(PATH)</code> , no profile	Set <code>(PATH)</code> , use absolute paths
<code>%</code> in crontab	Treated as newline	Escape as <code>\%</code>
Overlapping cron runs	Resource exhaustion, corrupt data	<code>flock -n</code>
Secrets in args or <code>set -x</code> logs	Visible in <code>(ps)</code> and logs	Env files with <code>(600)</code> , secret stores
CRLF line endings from Windows	<code>(bad interpreter: /bin/bash^M)</code>	<code>dos2unix script.sh</code> / <code>sed -i 's/\r\$//'</code>
Parsing <code>(ps)</code> / <code>(ls)</code> output	Fragile	Use <code>(pgrep)</code> , <code>(find)</code> , <code>(stat)</code> , <code>(/proc)</code>
Using <code>(kill -9)</code> first	Skips cleanup, leaves locks/corrupt state	<code>(kill)</code> (SIGTERM) first

Pitfall	Why it hurts	Fix
Editing <code>/etc/sudoers</code> or <code>sshd_config</code> blindly	Lockout	<code>visudo</code> , <code>sshd -t</code> , keep a second session open
Docker <code>RUN apt-get update</code> alone	Stale cached layer / bloated image	Combine update+install+clean in one <code>RUN</code>

18. Practice Exercises

Commands

- Using `find`, locate files over 50 MB modified in the last 7 days under `/var` and list them sorted by size.
- From an Nginx access log, produce the top 10 client IPs and the count of each HTTP status code (`awk`, `sort`, `uniq`).
- Use `comm` and process substitution to find packages installed on server A but missing on server B (`dpkg -l` / `rpm -qa`).
- Create a `systemd-run --scope` job limited to 256 MB RAM and 25% CPU; run a memory hog and watch it get constrained/killed.
- Reproduce inode exhaustion in a small loop-mounted filesystem; diagnose using `df -i` and fix it.
- Use `ss`, `lsof`, and `nc` to find a process on port 8080, then check reachability from another host.

Cron & systemd 7. Schedule a backup script at 02:00 daily with `flock`, a set `PATH`, and log redirection. Prove overlap prevention by making the script `sleep 120`. 8. Write a systemd unit for a small Python/Node app with a dedicated user, `Restart=on-failure`, `MemoryMax`, and sandboxing; score it with `systemd-analyze security`. 9. Convert the cron backup job into a systemd timer with `Persistent=true`; test by stopping the machine across the schedule. 10. Intentionally crash the service (`kill -9`) and watch it return; tune `RestartSec` and `StartLimitBurst`.

Scripting 11. Write `healthcheck.sh` that checks service state + HTTP status, restarts on failure, and sends a webhook. 12. Write `deploy.sh -v <version> [-n]` with `getopts`, dry-run, symlink releases, health check, and automatic rollback via `trap ERR`. 13. Write `provision.sh` that creates users from a CSV idempotently (re-running changes nothing). 14. Write a `retry()` function with exponential backoff and use it to wait for a database port (`nc -z`). 15. Run `shellcheck` on all your scripts, fix every warning, and add a `bats` test for one of them. 16. **Capstone:** build a `server-bootstrap.sh` that updates packages non-interactively, installs `nginx curl jq ufw`, creates a service user, configures UFW

(SSH/HTTP/HTTPS), installs a cron job and systemd service, and finishes with a self-test summary. Make it idempotent and `--dry-run` capable.

19. Notes & Corrections on the Source Handbook

While expanding the PDF, a few details were tightened for accuracy:

Item in source	Clarification
<code>sudo dnf check update</code>	The command is <code>dnf check-update</code> (hyphen). Note it returns exit code 100 when updates are available, which can trip <code>set -e</code> .
<code>sudo crontab -u deploy - e</code>	Should be <code>-e</code> with no space.
Cron log path <code>/home/vagrant/backups/backup.log</code>	Stray space — a path with a space breaks the redirection. Use <code>/home/vagrant/backups/backup.log</code> .
Unit file <code>After=network.target</code> with <code>Wants=network-online.target</code>	Pair <code>Wants=</code> with <code>After=network-online.target</code> so the service waits until the network is actually up.
<code>ProtectSystem=full</code>	<code>ProtectSystem=strict</code> (with <code>ReadWritePaths=</code>) is stricter and recommended when the service can be confined.
<code>apt-get upgrade -y</code> in automation	Consider <code>DEBIAN_FRONTEND=noninteractive</code> and <code>-o Dpkg::Options::="--force-confold"</code> to avoid config-file prompts.
<code>set -eo pipefail</code> in the backup script	Also add <code>-u</code> (<code>set -euo pipefail</code>) to catch unset variables, and quote paths.
<code>crontab -r</code>	Removes everything with no confirmation — back up first (<code>crontab -l > backup</code>) or use <code>crontab -ri</code> .
<code>dmesg -T grep -i oom</code>	May need <code>sudo</code> on hardened systems; also try <code>journalctl -k grep -i -E "oom killed process"</code> .

Remember: Automation is only as reliable as its failure handling. Quote variables, fail fast, log everything, make scripts idempotent, lock against overlap, and always test the unhappy path before it happens in production.